

HMARL-CBF 算法实现全说明（当前代码版）

本文档对应当前仓库 `hmarl_cbf/` 的实现，统一说明： - 代码结构 - 算法逻辑结构（上层/下层/环境/技能/同步协议） - 关键数学式子（动力学、奖励、MAPPO、QP/CBF/CLF、下层损失） - 默认配置与可切换项 - 训练与评测入口

1. 代码结构总览

主模块：`hmarl_cbf/`

- `env/`
 - `multi_uav_2d_env.py`: 2D 多智能体环境（双积分器、障碍、奖励、终止）
 - `lidar.py`: LiDAR 扫描模型
 - `observation.py`: 高层/低层观测拼装
- `skills/`
 - `library.py`: 技能定义（`turn_left/right`, `accelerate`, `decelerate`, `cruise`, `hover`）
 - `runtime.py`: 技能运行时（激活、计时、终止、内在奖励）
 - `termination.py`: 技能终止统一入口
- `policies/`
 - `high_level.py`: 上层离散技能策略 + value 网络
 - `low_level_qp.py`: 下层 QP 参数网络 + 低层 value 头
- `control/`
 - `constraint_builder.py`: 构建硬 CBF + 软 CLF + 输入限幅约束
 - `qp_solver.py`: 非可微 QP 求解（ECOS/SCS + fallback）
 - `diff_qp.py`: 可微 QP（KKT 反传路径）
 - `low_level_controller.py`: 下层控制链路（技能参考融合 + QP）
 - `sync_coordinator.py`: sync/async 技能切换协调器
- `high_level/`
 - `mappo.py`: 上层 MAPPO 更新器
- `buffer/`
 - `hier_rollout_buffer.py`: 分层回放（步级低层 + 轮次级高层）
- `train/`
 - `trainer_sync_onpolicy.py`: 主训练器（rollout/update/eval）
 - `run_sync_onpolicy.py`: 训练入口
 - `eval_checkpoint_random.py`: 随机场景评测入口
- `types.py`
 - 数据结构定义（状态、观测、QP、transition 等）

配置目录: configs/hmarl_cbf/

- default_async_onpolicy_gcbfplus.yaml (当前常用)
- default_sync_onpolicy.yaml
- default_sync_onpolicy_gcbfplus.yaml

2. 任务与系统建模

2.1 状态与控制

每个智能体状态 (2D 双积分器) :

$$x_i = [p_x, p_y, v_x, v_y]$$

控制输入:

$$u_i = [a_x, a_y]$$

环境离散更新 (见 env/multi_uav_2d_env.py) :

$$v_{t+1} = \text{clip}(v_t + u_t dt, -v_{\max}, v_{\max})$$

$$p_{t+1} = p_t + v_{t+1} dt$$

并对动作先做 action_limit 限幅。

2.2 终止条件

单智能体到达条件:

$$\|p - p_{\text{goal}}\| \leq \text{goal_threshold}$$

且

$$\|v\| \leq \text{goal_speed_threshold}$$

episode 终止: - 全体到达, 或 - terminate_on_collision=true 且任意不安全 (碰撞或出界), 或 - 到达 horizon (truncated)。

3. 观测结构 (部分可观测)

每个智能体高层观测 obs_high: - self_state = [p_x, p_y, v_x, v_y] - goal_relative = goal - position - neighbor_summary (最多 max_neighbors, 每个邻居 4 维: 相对位置 2 + 相对速度 2)

每个智能体低层观测 obs_low: - self_state - goal_relative - lidar_scan (N_beam 束, 归一化到 [0,1]) - neighbor_summary

LiDAR 模型 (env/lidar.py) : - 每束做射线-圆障碍/邻居求交, 取最近距离 - 命中返回 max_range - 可选高斯噪声: $\mathcal{N}(0, \sigma^2)$

4. 技能层 (7 元组映射)

实现技能: - turn_left, turn_right, accelerate, decelerate, cruise, hover

SkillSpec 含 7 元组字段: - 启动集函数 initiation_set_fn - 中止集函数 termination_set_fn - 最大时长 max_duration - 中止函数 termination_fn(state, ctx, tau) - 安全集函数 safety_constraints_fn - 内在奖励函数 intrinsic_reward_fn - 安全技能策略 (参考动作) safe_skill_policy

技能终止逻辑:

$$\beta_i = \text{termination_fn}(\cdot)$$

通常实现为:

$$(\text{in termination_set}) \vee (\tau \geq \text{max_duration})$$

5. 分层控制与执行协议

5.1 总体结构

每步链路: 1. 上层 (按轮次) 给技能 z_i 2. 下层网络输入 (obs_low, z_i) 输出 QPParam 3. 技能策略输出 $u_{\text{ref_skill}}$ 4. 融合参考 5. 构建 QP 约束 (CBF/CLF/输入限幅) 6. 求解 QP 得执行动作 u

融合参考:

$$u_{\text{ref}}^{\text{fused}} = w_{\text{skill}} u_{\text{ref}}^{\text{skill}} + (1 - w_{\text{skill}}) u_{\text{ref}}^{\text{policy}}$$

5.2 sync/async 切换 (sync_coordinator.py)

mode=sync:

$$\text{sync_switch} = \text{all}(\beta_i) \vee \max(\tau_i) \geq T_{\text{sync_max}}$$

mode=async: - 智能体 i 若 $\beta_i = 1$ 或 $\tau_i \geq T_{\text{sync_max}}$ 则单独切换技能 - 未触发切换的智能体保持当前技能

6. 上层: MAPPO (离散技能)

策略网络 (policies/high_level.py) : - backbone: MLP(Tanh) -> actor logits + value - 动作为离散技能 id (Categorical)

MAPPO 更新 (high_level/mappo.py) :

$$r = \exp \left(\log p_{\text{new}} - \log p_{\text{old}} \right)$$

$$L_{\text{actor}} = -\mathbb{E} \left[\min \left(rA, \text{clip} \left(r, 1 - \epsilon, 1 + \epsilon \right) A \right) \right]$$

$$V_{\text{clip}} = V_{\text{old}} + \text{clip} \left(V_{\text{new}} - V_{\text{old}}, -\epsilon, \epsilon \right)$$

$$L_{\text{value}} = \frac{1}{2} \mathbb{E} \left[\max \left(\left(V_{\text{new}} - R \right)^2, \left(V_{\text{clip}} - R \right)^2 \right) \right]$$

总损失:

$$L_{\text{high}} = L_{\text{actor}} + c_v L_{\text{value}} - c_{\text{ent}} H$$

高层样本来自高层 option 片段: (obs_high, z, logp, value, return_ext, advantage, value_target)。

7. 下层: 参数化 QP + 可微求解

7.1 下层网络输出

low_level_qp.py 输出: - u_ref (2维) - r_diag (二次项对角, softplus 保正) - w_clf (CLF 松弛权重, softplus 保正) - cbf_k0, cbf_k1 (CBF 增益, softplus 保正) - clf_k (CLF 增益, softplus 保正) - 低层 value 头: low_value(obs_low, skill_id)

7.2 QP 标准型

constraint_builder.py 构建:

$$\min_{u, \delta} \frac{1}{2} u^T \text{diag} \left(r_{\text{diag}} \right) u + f^T u + w_{\text{clf}} \delta$$

当

$$f = -\text{diag} \left(r_{\text{diag}} \right) u_{\text{ref}}$$

时, 目标等价于

$$\frac{1}{2} \| u - u_{\text{ref}} \|_R^2 + w_{\text{clf}} \delta$$

约束:

$$A_{\text{cbf}} u \leq b_{\text{cbf}}$$

$$A_{\text{clf}} u \leq b_{\text{clf}} + \delta$$

$$u_{\text{min}} \leq u \leq u_{\text{max}}$$

$$\delta \geq 0$$

7.3 CBF 形式 (可切换)

配置键: cbf_mode

1. distributed_ecbf

$$h = \|p_{\text{rel}}\|^2 - d_{\text{safe}}^2, \quad \dot{h} = 2p_{\text{rel}}^\top v_{\text{rel}}, \quad \text{const} = 2\|v_{\text{rel}}\|^2$$

线性行:

$$A = -2p_{\text{rel}}, \quad b = \text{const} + k_1 \dot{h} + k_0 h$$

2. distributed_gcbfplus

$$h_0 = \|p_{\text{rel}}\|^2 - d_{\text{safe}}^2, \quad \dot{h}_0 = 2p_{\text{rel}}^\top v_{\text{rel}}$$

$$h_1 = \dot{h}_0 + \alpha_0 h_0, \quad \alpha_0 = k_0$$

$$L_f(h_1) = 2\|v_{\text{rel}}\|^2 + 2\alpha_0(p_{\text{rel}}^\top v_{\text{rel}})$$

责任分配约束:

$$A = -2p_{\text{rel}}, \quad b = \text{share}(L_f(h_1) + \alpha_1 h_1), \quad \alpha_1 = k_1$$

3. distributed_hocbf54

$$h = \sqrt{4u_{\text{max}}(\|p_{\text{rel}}\| - d_{\text{safe}})} + n^\top v_{\text{rel}}, \quad n = \frac{p_{\text{rel}}}{\|p_{\text{rel}}\|}$$

对应线性行 $Au \leq b$ 由 `_build_hocbf54_row / _build_hocbf54_row_torch` 实现。

7.4 CLF 形式

期望速度:

$$v_{\text{des}} = \begin{cases} \text{clf_v_des_vector}, & \text{技能提供该向量} \\ s_{\text{ref}} \cdot \text{goal_dir}, & \text{否则} \end{cases}$$

$$V = \frac{1}{2} \|v - v_{\text{des}}\|^2, \quad A_{\text{clf}} = (v - v_{\text{des}})^\top, \quad b_{\text{clf}} = -k_{\text{clf}} V$$

CLF 在 QP 中通过松弛变量 δ 软化。

7.5 求解器策略

推理 QP (`qp_solver.py`): - 先 ECOS - 失败则 SCS - 再失败用 deterministic stub (按目标项闭式 + 限幅)

可微 QP (diff_qp.py) 用于训练反传： - 同样 ECOS -> SCS -> 张量 fallback - 保留 r_{diag} / w_{clf} / cbf_k^* / clf_k / u_{ref} 的梯度路径

8. 下层训练模式（可切换）

配置键：train.low_update_mode

8.1 target_regression

$$L_{low} = \frac{1}{2} \| u_{qp} - u_{target} \|^2$$

$$u_{target} = u_{exec} + scale \cdot advantage \cdot goal_{dir}$$

并可裁剪到输入边界。

8.2 onpolicy_ppo

rollout 时： - 先计算 QP 均值动作 μ - 采样动作： $a_{sample} = \mu + \epsilon, \epsilon \sim \mathcal{N}(0, \sigma^2)$
- 记录 $\log p_{old}$ 与 $value_{old}$

更新时：

$$r = \exp \left(\log p_{new} - \log p_{old} \right)$$

$$L_{actor} = - \min \left(rA, \text{clip}(r, 1 - \epsilon, 1 + \epsilon) A \right)$$

$$L_{value} = \frac{1}{2} (V - R)^2$$

$$L_{low} = L_{actor} + c_v L_{value} - c_{ent} H$$

默认低层回报以内在奖励为主 (low_ext_reward_coef=0.0)：

$$low_return = r_{int} + coef \cdot r_{ext}$$

9. 奖励、成本与指标

9.1 环境外在奖励 reward_ext

$$progress = d_{prev} - d_{curr}$$

$$r = w_{progress} \cdot progress - w_{time}$$

到达加 reach_bonus, 碰撞减 collision_penalty, 出界减 oob_penalty。

9.2 内在奖励 reward_int

在 `skills/library.py` 中定义，共性项包括：- 加速度惩罚 - 转向惩罚 - 速度偏差惩罚 - 航向偏差惩罚 - 小权重前进奖励

各技能再附加对应 bonus/penalty（如 hover 的静止与贴近目标奖励）。

9.3 安全成本

$$c_i = \mathbf{1}\{\text{collision or out-of-bounds}\}$$

在 `env.info["costs"]` 输出。

9.4 常用日志

- `episode_return_mean`（外在回报均值）
- `safe_reach_ratio`
- `eval_success_rate`, `eval_collision_rate`, `eval_reach_rate`
- `eval_qp_feasible_rate`
- `loss_high_*`, `loss_low_*`

10. 默认配置

(`default_async_onpolicy_gcbfplus.yaml`)

关键默认值：- 环境： `n_agents=4`, `n_obstacles=3`, `dt=0.1`, `horizon=200` - 限幅：
`action_limit=2.0`, `velocity_limit=2.0` - 感知： `lidar_beams=32`, `lidar_range=4.5`,
`neighbor_radius=3.0` - 协议： `mode=async`, `t_sync_max=20` - 安全： `d_min_agent=0.6`,
`d_safe_obs=0.6` - 技能： `ref_speed=1.2`, `decelerate_target_speed=0.3` - CBF：
`cbf_mode=distributed_gcbfplus`, `cbf_share_agent=0.5`, `cbf_share_obs=1.0` - 约束：
`use_input_bounds=true` - 训练： `rollout_steps=200`, `eval_interval=10` - 折扣：
`gamma_high=0.99`, `lam_high=0.95`, `gamma_low=0.99` - 下层：
`low_update_mode=target_regression`（可切到 `onpolicy_ppo`）

11. 训练与评测入口

11.1 训练

入口： `python -m hmarl_cbf.train.run_sync_onpolicy`

示例（异步 + GCBF + 下层 PPO）：

```
python -m hmarl_cbf.train.run_sync_onpolicy \
  --config configs/hmarl_cbf/default_async_onpolicy_gcbfplus.yaml \
```

```
--low-update-mode onpolicy_ppo \  
--run-name train_async_lowppo_gcbfplus
```

11.2 评测

入口: `python -m hmarl_cbf.train.eval_checkpoint_random`

```
python -m hmarl_cbf.train.eval_checkpoint_random \  
--checkpoint artifacts/hmarl_cbf/<run>/checkpoints/last.pt \  
--episodes 100 \  
--seconds 30 \  
--deterministic \  
--run-name eval_100x30s
```

输出: - `episode_results.csv` - `summary.json`

12. 实现注意事项

1. 世界单位是仿真单位（数值尺度），默认不显式绑定真实米制。
2. 邻居/障碍 CBF 采用“感知到才加约束”：
 - 邻居由 `neighbor_radius` 过滤。
 - 障碍由 `lidar_range`（表面距离）过滤。
3. 当前版本 QP 不可解时不会强制“减速 fallback”；推理求解器走内部 fallback。
4. 上层回报用外在奖励聚合；下层回报默认以内在此奖励为主（`low_ext_reward_coef=0.0`）。

13. 一句话总结

当前实现是可切换 sync/async 的分层多智能体安全控制框架：上层 MAPPO 技能切换，下层网络学习 QP 参数并通过 CBF/CLF-QP 逐步生成安全动作，支持 GCBF+ 风格分布式手工 CBF 与下层 on-policy PPO 路径。